

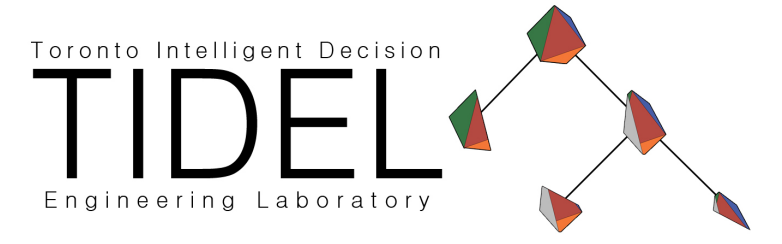
Intruder Alert! Optimization Models for Solving the Mobile Robot Graph-Clear Problem

Michael Morin^{a,b} Margarita P. Castro^{b,*} Kyle E. C. Booth^{b,*} Tony T. Tran^b Chang Liu^b J. Christopher Beck^b

^{*}M.P. Castro and K.E.C. Booth are equality contributing authors.

^a Department of Operations and Decision Systems, Université Laval, Québec, QC, Canada

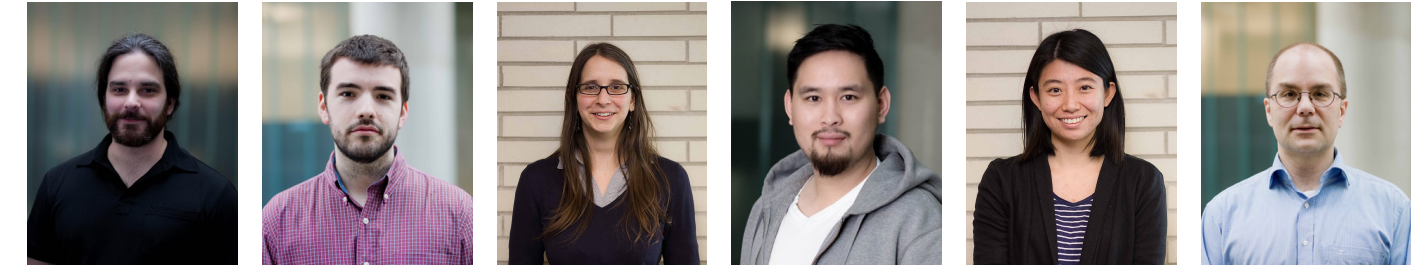
^b Department of Mechanical and Industrial Engineering, University of Toronto, Toronto, ON, Canada



UNIVERSITY OF
TORONTO



UNIVERSITÉ
LAVAL



Keywords Pursuit-evasion; Graph-clear problem; Constraint programming; Mixed-integer linear programming; Optimization; Mobile robotics

Context of the Graph-Clear Problem

Objective of Graph-Clear

The objective is to find a *schedule* minimizing the total number of robots (agents) needed to clear a discrete environment from all intruders. [1]

Practical Assumptions

- ▶ Multiple homogeneous robots
- ▶ Perfect detection within range
- ▶ Worst-case assumption on the intruders' motion

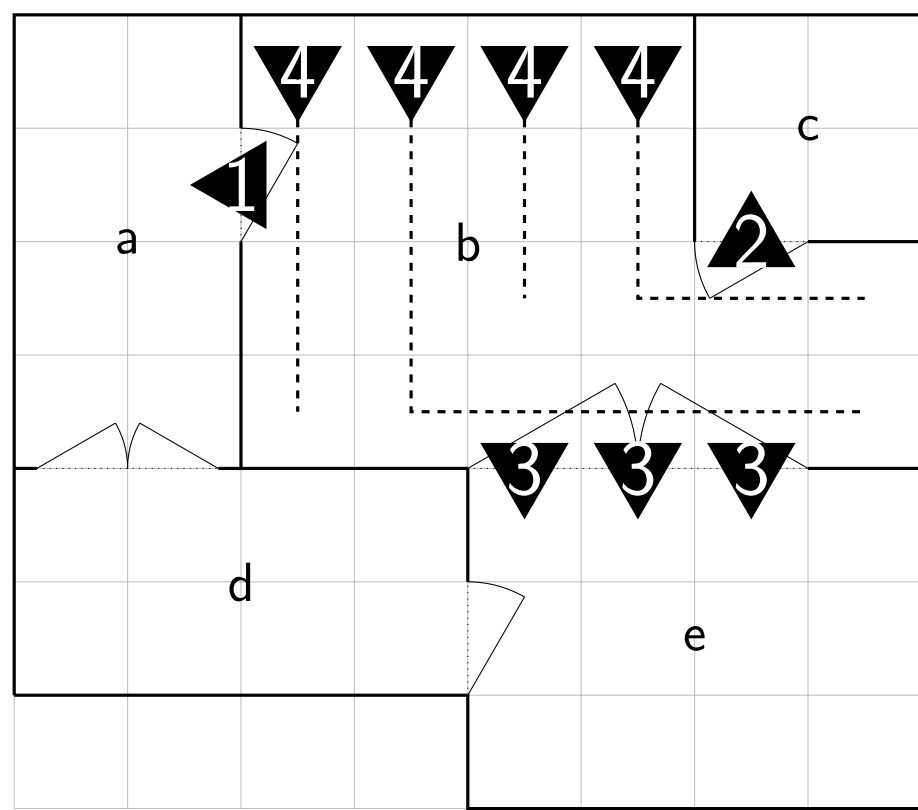
Applications

- ▶ Surveillance
- ▶ Search and rescue
- ▶ Building security

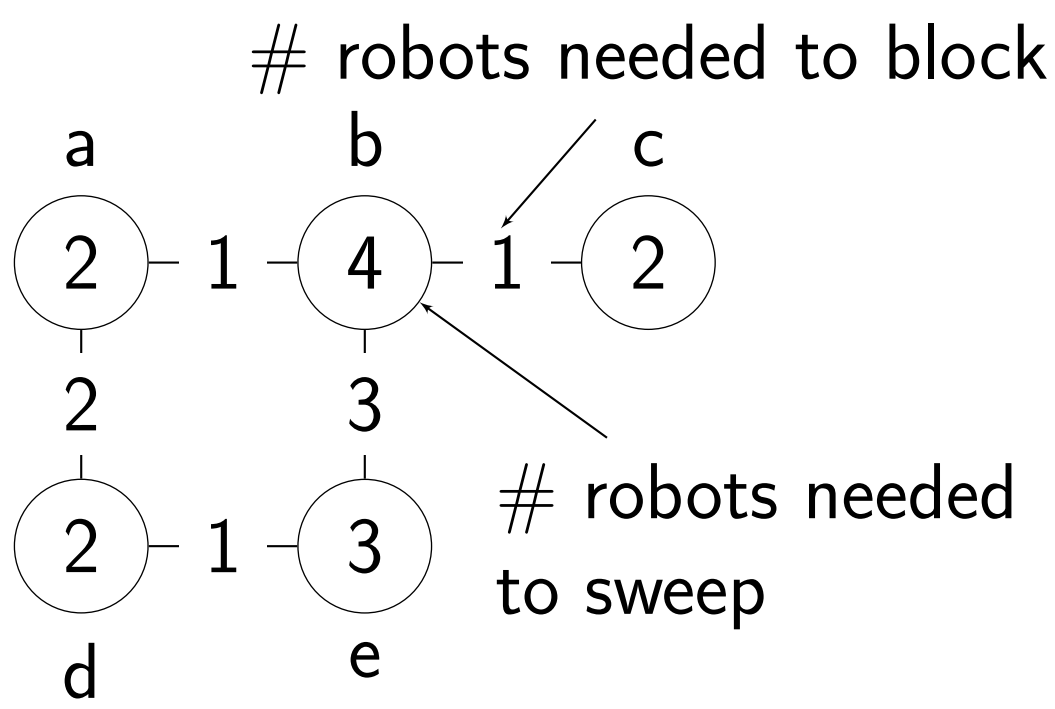
Graph-Clear Problem: Definition and Examples

Suppose we need to make sure room b does not contain any intruder.

1. 1 robot guards the door to room a
 2. 1 robot guards the door to room c
 3. 3 robots guard the door to room e
 4. 4 robots sweep room b
- 9 robots are needed to clear b.



Problem instance



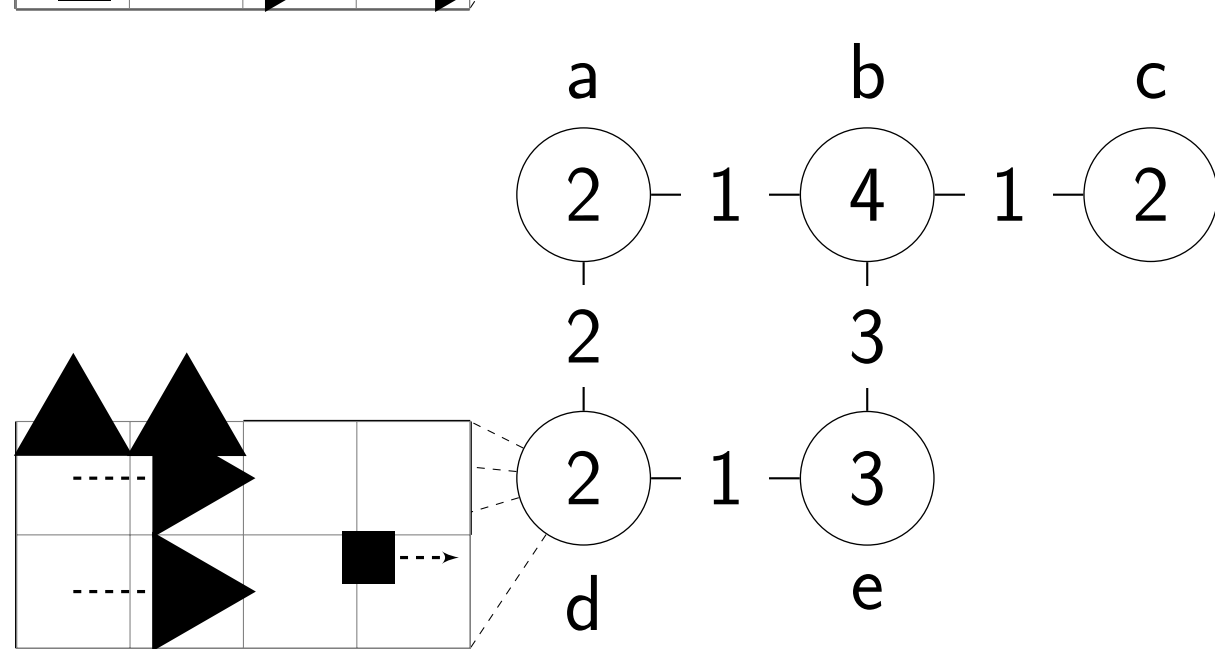
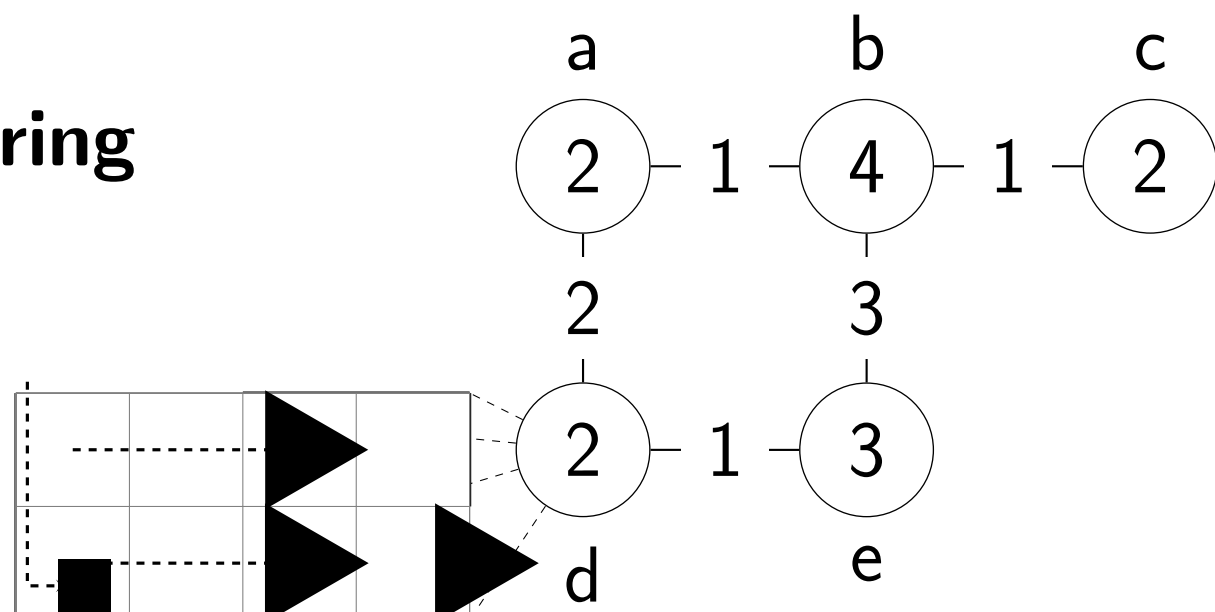
A problem instance is a *weighted graph* $G = (V, E, a, b)$ where $a : V \rightarrow \mathbb{N}^+$ and $e : E \rightarrow \mathbb{N}^+$ are the weights over the nodes and edges.

Recontamination and Clearing

A *contaminated* node (or edge) might contain an intruder, otherwise it is *cleared*.

An *open path* (recontamination path) between two nodes (or edges) does not contain any blocked edge.

The *recontamination* of a node (or edge) occurs instantly via open paths.



Recontamination of (top) and from (bottom) room d while sweeping

Solution Characteristics

A *progressive* solution does not allow recontamination.

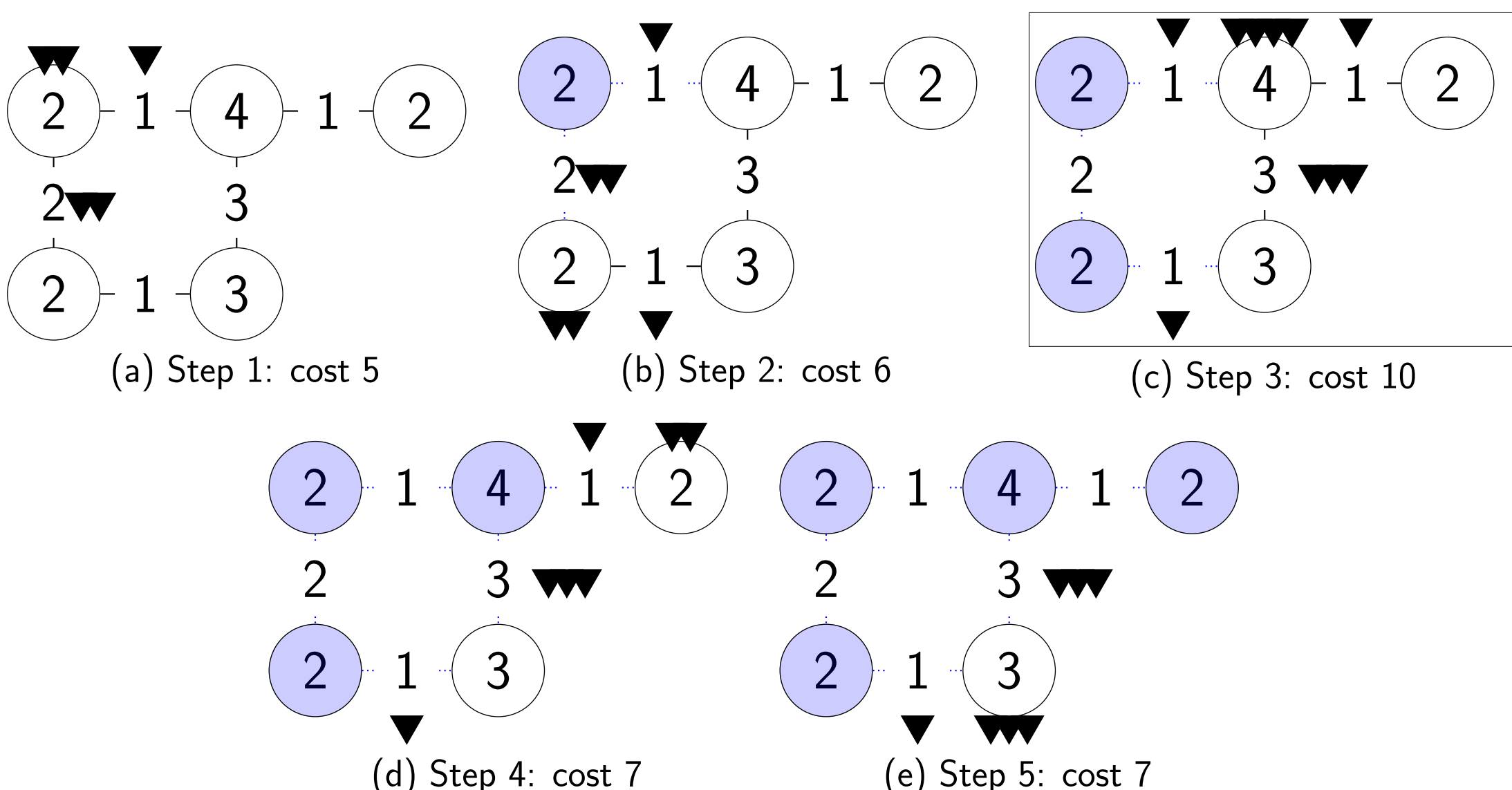
In a *contiguous* solution, at each step, the cleared rooms form a connected component.

Assumptions

1. Solutions are progressive
 2. A single node, $v \in V$, is swept at each step
 3. No extra edges are blocked beyond those to assure no recontamination
- Under these assumptions, it is sufficient to define a solution in terms of the node swept at each step.

Solution (Schedule)

The *cost* of a schedule = the maximal # of robots needed at any step



Theoretical Results

Is there always an optimal, progressive solution? (open)

Progressive Solutions – Contributions

We identified 3 specific recontamination cases:

- ▶ *single-node* recontamination
- ▶ *complete/partial* recontamination of a connected component

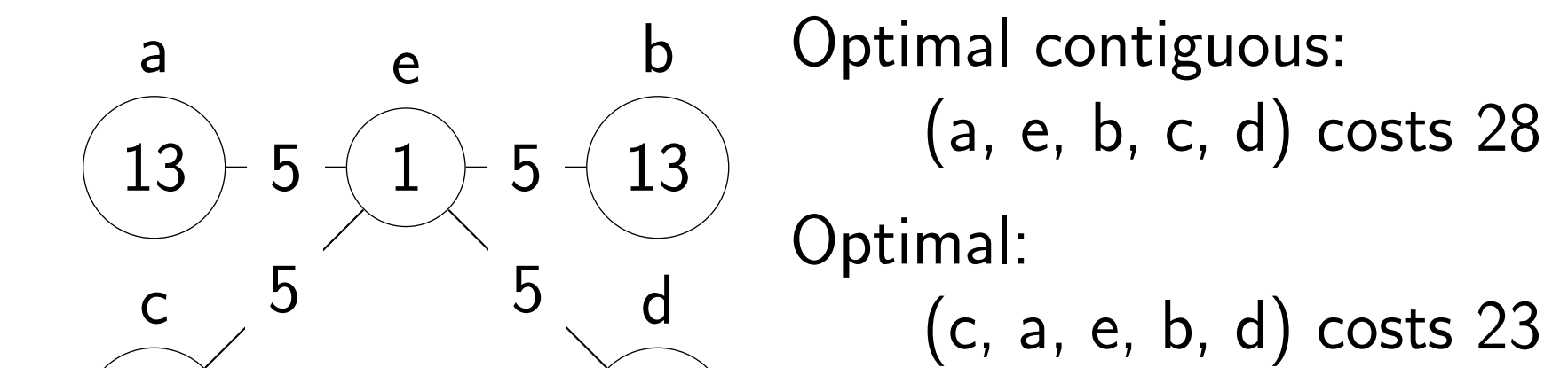
Our main results:

- ▶ *single-node/complete* recontamination steps cannot improve solution quality

The case for *partial* recontamination remains open.

Might enforcing contiguity remove all optimal solutions?

Contiguous Solutions – Contributions



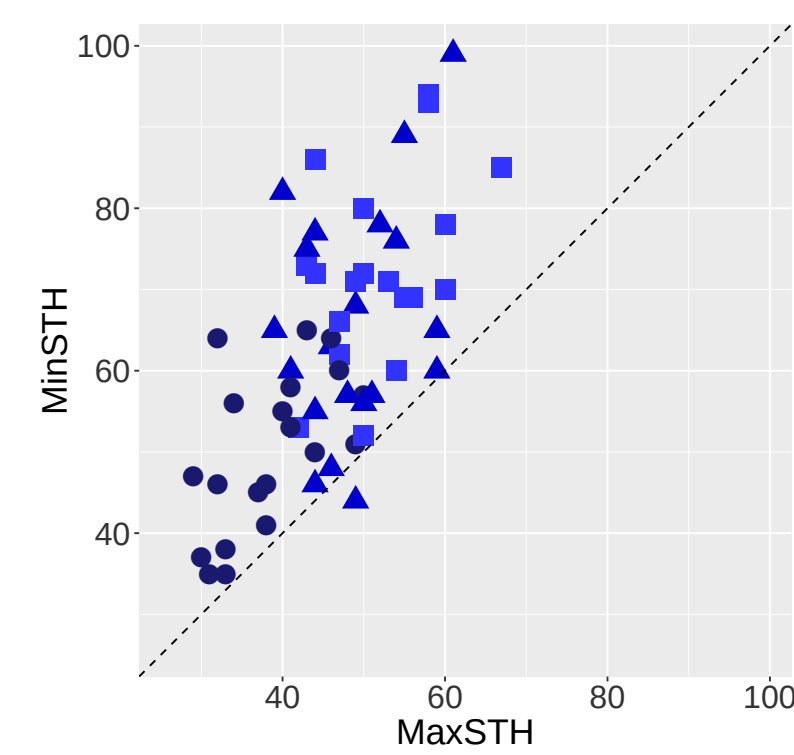
Heuristics

Main heuristic form from the literature [2] (3 steps):

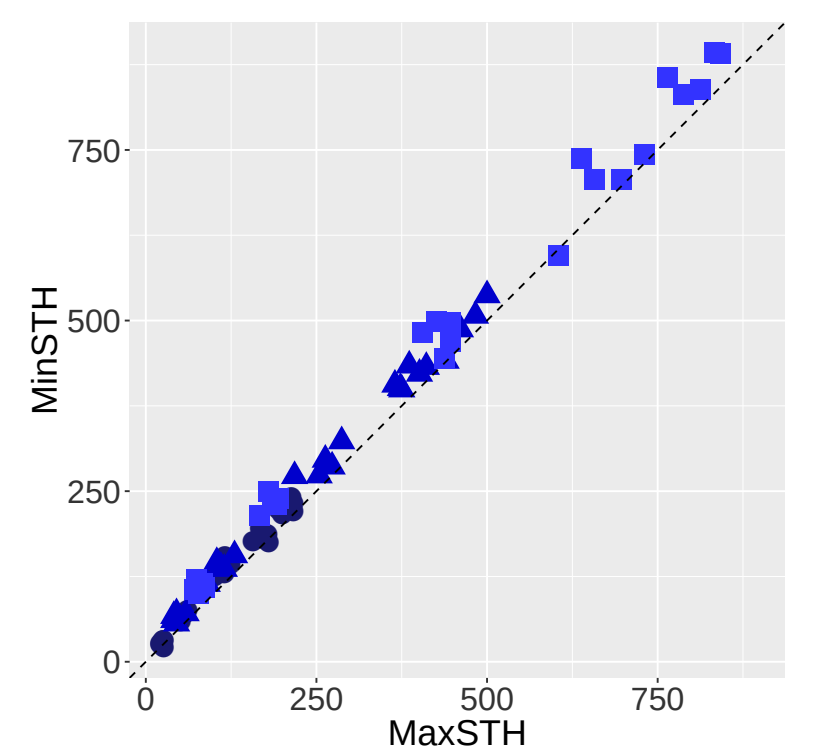
1. Transform the graph G into a (*minimum*) spanning tree T by removing edges
2. Find a schedule that clears T
3. Construct a solution for G using the one found on T by guarding the necessary edges

Contribution

Maximum spanning trees (ST) are best (w.r.t. minimum ST) on our benchmark instances.



60 random planar graphs
(# of robots in schedule)



75 random graphs
(# of robots in schedule)

No. of nodes
● 20
▲ 30
■ 40

Exact Methods

Contributions: 3 Models plus Evaluation

- ▶ $H = \{1, \dots, n\}$ is the set of steps
- ▶ Z is the objective function value
- ▶ a_v is the # of robots needed to sweep $v \in V$
- ▶ b_e is the # of robots needed to block $e \in E$
- ▶ c_{sweep} (resp. c_{block}) is the highest possible sweeping (resp. blocking) cost
- ▶ $\delta(X)$, with $X \subset V$, is the subset of edges with a node in X and a node in $X \setminus V$

CPS Idea: Model the sequence of nodes that are swept using the $W[t]$ decision variables ($t \in H$)

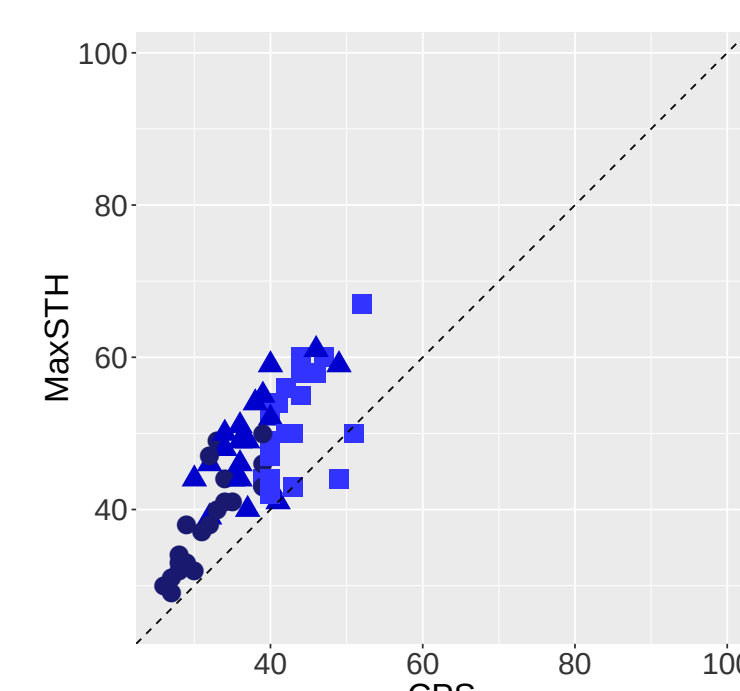
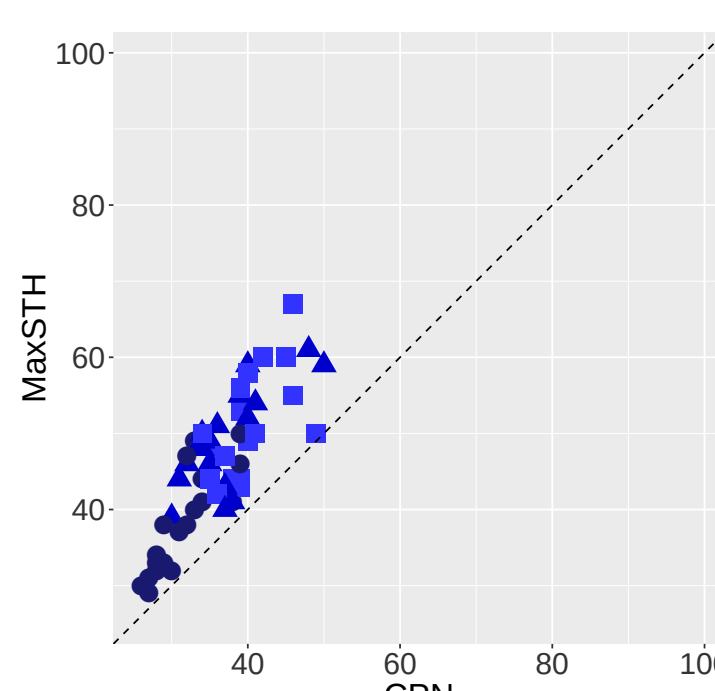
$$\begin{aligned} \min Z \\ \text{s.t. } \text{ALLDIFFERENT}(W) \\ Z &\geq a_{W[t]} + \sum_{e \in \delta(W[t])} b_e + \sum_{e \in \delta(\{W[1], \dots, W[t-1]\}) \setminus \delta(W[t])} b_e \quad \forall t \in H \\ W[t] &\in V \quad \forall t \in H \\ \underline{Z} &\leq Z \leq \bar{Z}, Z \in \mathbb{Z}^+ \end{aligned}$$

CPN Idea: Model the step at which each node/edge is swept/blocked and released using the $T[i]$, $L[i, j]$, and $U[i, j]$ dec. var. ($i, j \in V$)

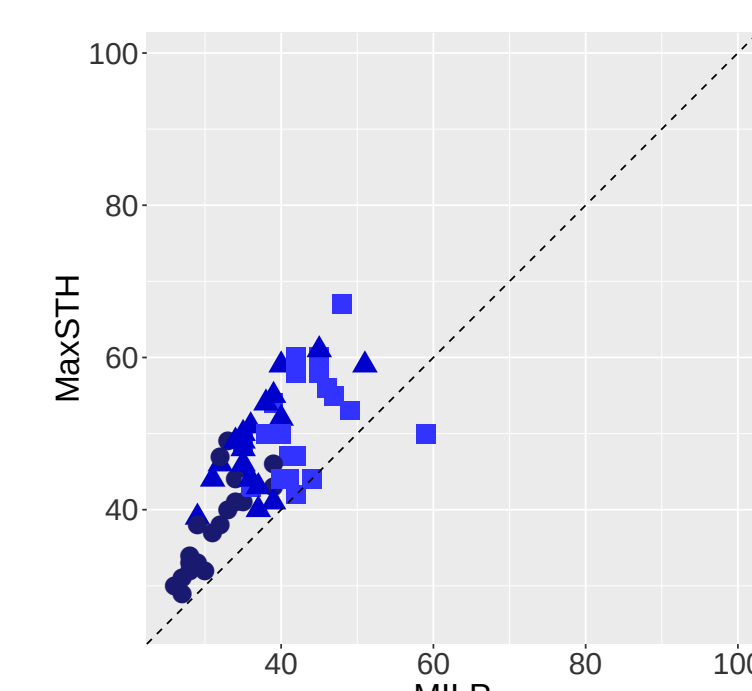
$$\begin{aligned} \min Z \\ \text{s.t. } Z &= \max_{t \in H} \{S[t] + B[t]\} \\ T[i] = t &\Rightarrow S[t] = a_i + \sum_{e \in \delta(i)} b_e \quad \forall i \in V, t \in H \\ \text{ALLDIFFERENT}(T) \\ T[i] < T[j] &\Rightarrow L[i, j] = T[i] \quad \forall \{i, j\} \in E \\ T[i] < T[j] &\Rightarrow U[i, j] = T[j] \quad \forall \{i, j\} \in E \\ (L[i, j] \leq t \wedge U[i, j] \geq t) &\Rightarrow I[i, j, t] = 1 \quad \forall \{i, j\} \in E, t \in H \\ \wedge (T[i] \neq t \wedge T[j] \neq t) &\Rightarrow I[i, j, t] = 0 \quad \forall \{i, j\} \in E, t \in H \\ B[t] &= \sum_{\{i, j\} \in E} b_{\{i, j\}} I[i, j, t] \quad \forall t \in H \\ T[i], L[i, j], U[i, j] &\in H \quad \forall i \in V, \{i, j\} \in E \\ S[t] &\in \{1, \dots, c_{\text{sweep}}\} \quad \forall t \in H \\ B[t] &\in \{1, \dots, c_{\text{block}}\} \quad \forall t \in H \\ I[i, j, t] &\in \{0, 1\} \quad \forall i, j \in V, t \in H \\ \underline{Z} &\leq Z \leq \bar{Z}, Z \in \mathbb{Z}^+ \end{aligned}$$

MILP Idea: Model the sequence of nodes that are swept using the $X[i, t]$ decision variables ($i \in V, t \in H$)

$$\begin{aligned} \min Z \\ \text{s.t. } Z &\geq \sum_{i \in V} a_i X[i, 1] + \sum_{\{i, j\} \in E} b_{\{i, j\}} Y[i, j, 1] \\ Z &\geq \sum_{i \in V} a_i (X[i, t] - X[i, t-1]) \\ &+ \sum_{\{i, j\} \in E} b_{\{i, j\}} Y[i, j, t] \quad \forall t \in H \setminus \{1\} \\ \sum_{i \in V} X[i, t] &= t \quad \forall t \in H \\ X[i, t] &\leq X[i, t+1] \quad \forall i \in V, \forall t \in H \setminus \{n\} \\ X[i, t] - X[j, t] &\leq Y[i, j, t] \quad \forall \{i, j\} \in E, t \in H \\ X[j, t] - X[i, t] &\leq Y[i, j, t] \quad \forall \{i, j\} \in E, t \in H \\ X[i, t] - X[i, t-1] &\leq Y[i, j, t] \quad \forall \{i, j\} \in E, t \in H \setminus \{1\} \\ X[i, t] &\in \{0, 1\} \quad \forall i \in V, t \in H \\ Y[i, j, t] &\in \{0, 1\} \quad \forall \{i, j\} \in E, t \in H \\ \underline{Z} &\leq Z \leq \bar{Z}, Z \in \mathbb{R}^+ \end{aligned}$$



60 random planar graphs
(# of robots in schedule)



Conclusion

- ▶ Single-node and complete recontamination cannot improve solution quality
- ▶ Enforcing contiguity can remove all optimal solutions
- ▶ The proposed optimization-based techniques are currently the most suitable for solving the graph-clear problem

References

- [1] Andreas Kolling and Stefano Carpin. "Pursuit-evasion on trees by robot teams". In: *IEEE Transactions on Robotics* 26.1 (2010), pp. 32–47. DOI: 10.1109/TRD.2009.2035737.
- [2] Andreas Kolling and Stefano Carpin. "The graph-clear problem: definition, theoretical properties and its connections to multirobot aided surveillance". In: *Proceedings of the IEEE/RSS International Conference on Intelligent Robots and Systems*. 2007, pp. 1003–1008. DOI: 10.1109/IRROS.2007.4399368.
- [3] Michael Morin, Margarita P. Castro, Kyle E.C. Booth, Tony T. Tran, Chang Liu, and J. Christopher Beck. "Intruder alert! Optimization models for solving the mobile robot graph-clear problem". In: *Constraints* 23.3 (2018), pp. 335–354. DOI: 10.1007/s10601-018-9288-3.